03 — Field validation

What it detects: internally inconsistent documents — the kind of arithmetic or date mistake that falsified or hand-edited documents often contain. It is **pure rule logic over one document's own extracted fields**: no I/O, no model, no database.

Code: `services/field_validation_service.py` (`validate_fields`), wrapped by `tasks/document_checks.py` (`run_document_checks`), which stores a `document_checks` row with `check_type = field_validation`. Highlight regions come from `services/field_regions.py` and `services/field_exception_service.py` .

Sub-checks

`validate_fields(extracted_fields) -> {"result": "pass"|"flag", "details": {...}}` runs these independent sub-checks; the overall result is `flag` if **any** sub-check is `flag` .

Sub-check	Flags when	Skipped when
`date_in_future`	the document's `date` core field is after today	no parseable date
`line_item_arithmetic`	a line's quantity × unit price (− its discount) ≠ the line total printed	no line with quantity, unit price and total
`subtotal_line_item_consistency`	the line items that count toward the total don't add up to the `subtotal` — or, with no subtotal shown, to the `amount` (less `tax_amount` when tax is shown)	no line totals, or no subtotal/total
`total_tax_consistency`	`amount` ≠ `subtotal + tax_amount` (or ≠ `subtotal` when no tax line is shown)	no subtotal
`tax_rate_consistency`	`tax_amount` ≠ taxable base × `tax_rate` (base = subtotal, or total less tax)	no rate or no tax amount
`amount_in_words_consistency`	the amount written in words ≠ the total (or the subtotal) in figures	no amount in words, or it can't be read
`reference_number_format`	the reference number is not a plausible identifier (regex `^[A-Za-z0-9](?:[A-Za-z0-9\-/_. #]*[A-Za-z0-9])?$`)	—

Calculation checks

Extraction returns structured `line_items` (description, quantity, unit price, discount, line total, and whether the line counts toward the total) and `amount_in_words` (the words and the number they spell), stored in `documents.extracted_fields` . The model is told to copy every number **exactly as printed and never recompute** one: the discrepancies are the point. On a real scanned invoice it returned `3 × 6,000 = 16,600` as printed, and all three errors on that page were flagged.

- **Amounts are compared to the cent** (one cent of rounding per term, plus half a cent per unit when a unit price is multiplied out), not by a percentage. A 1 % allowance on a 55,500 total would hide a 400 discrepancy.
- **Excluded lines:** a line the document marks as already paid or informational doesn't count toward the total. If including it is what makes the sum match, the check passes rather than flagging the model's judgment.
- **Amount in words:** English words are read by code (`services/amount_words.py`), not taken from the model, which tends to "correct" words that contradict the figures. Other languages (Arabic) use the model's reading. Words that leave out the fils/cents still match.
- Each wrong line total and the amount-in-words line are located on the page (`services/field_locator_service.py`) and drawn as field-exception boxes.

Documents extracted before structured line items existed fall back to the old heuristic for `subtotal_line_item_consistency`: additional fields whose name suggests a per-line total (name contains `item`, value hint `total` / `amount`), compared with the subtotal only.

Output shape (real example)

```
{
  "result": "flag",
  "details": {
    "date_in_future": {"status": "pass", "reason": "Document date 2026-08-25 is not in the future."},
    "total_tax_consistency": {
      "status": "flag",
      "reason": "Total 197500.00 does not match subtotal 180000.00 + tax 9000.00 (expected 189000.00).",
      "regions": [
        {"field": "amount", "label": "Total amount", "value": "197,500.00 AED",
         "caption": "Field exception: Total amount (197,500.00 AED vs expected 189,000.00)",
         "bounding_box": {"page": 1, "x": 0.7821, "y": 0.5081, "width": 0.1321, "height": 0.0119}}
      ]
    },
    "reference_number_format": {"status": "pass", "reason": "…"},
    "subtotal_line_item_consistency": {"status": "pass", "reason": "Subtotal matches the sum of 3 line-item
field(s)."}
  }
}
```

Unlike the forensic checks, `details` is a **dict keyed by sub-check**, not a list of findings. Each sub-check is `{status, reason, regions?}` where `status` is `pass`, `flag` or `skipped` (no value to check, or unparseable); `regions` (only on `flag`) are drawn as **purple field exception** boxes over the PDF. If a field's position could not be located, the exception is still reported without a box. Rows stored before locations existed are re-derived at read time (`with_field_regions`) without modifying the stored row.

Known limitations

- Depends entirely on extraction quality; a wrong extracted value produces a wrong flag (or a miss).
- Amounts must agree to the cent (see above). A document that rounds each line differently from its printed total by more than a cent per term will flag.
- Line items and the amount in words come from the extraction model. A line it fails to list, or a number it misreads, produces a miss or a wrong flag; the reason text always shows the numbers used.
- `date_in_future` compares against `date.today()` on the worker's machine, not the case's submission date, so a worker with a wrong clock can mis-flag.

Risk rules fed

rule_id	Sub-check	Weight	Severity
`field.date_in_future`	`date_in_future`	25	high
`field.line_item_arithmetic_mismatch`	`line_item_arithmetic`	25	high
`field.subtotal_line_item_mismatch`	`subtotal_line_item_consistency`	15	medium
`field.total_tax_mismatch`	`total_tax_consistency`	20	high

rule_id	Sub-check	Weight	Severity
field.tax_rate_mismatch	tax_rate_consistency	15	medium
field.amount_in_words_mismatch	amount_in_words_consistency	25	high
field.reference_number_malformed	reference_number_format	6	low

(Weights are the template defaults; each company's Reviewer L2s can change them for their company — see 11.)